

KAFKA

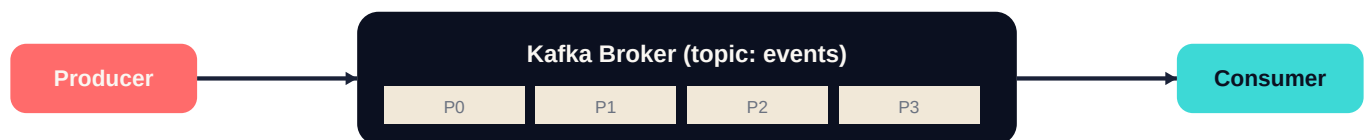
× PYTHON

an elegant, friendly cheatsheet for streaming devs

[producers](#)[consumers](#)[topics](#)[partitions](#)[offsets](#)

◆ what is kafka?

A distributed event-streaming platform — think of it as a durable, replayable, append-only log that many apps share.



■ install

```
# pick your client
pip install kafka-python      # pure-python, simple
pip install confluent-kafka   # C-based, blazing fast
pip install aiokafka          # asyncio native
```

🔍 tiny glossary

- Topic — named stream of events
- Partition — ordered log inside a topic
- Offset — position of a record in a partition
- Producer — sends events · Consumer — reads events
- Group — consumers sharing the work of a topic

? which client should i use?

kafka-python

pure python · easy to debug · great for learning

confluent-kafka

fastest · production grade · uses librdkafka

aiokafka

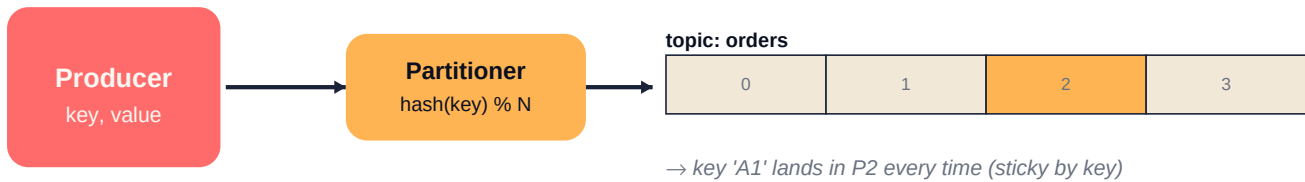
async/await · perfect for FastAPI & async services



Producers

send events to a topic — fire & forget, sync, or async

➤ how a producer writes



```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    key_serializer=lambda k: k.encode('utf-8') if k else None,
    acks="all",          # 0 / 1 / "all"
    linger_ms=20,        # batch a bit
    compression_type="zstd",
)

future = producer.send(
    "orders",
    key="user-42",
    value={"item": "coffee", "qty": 2},
)
```

```
from confluent_kafka import Producer
import json

p = Producer({
    "bootstrap.servers": "localhost:9092",
    "acks": "all",
    "linger.ms": 20,
    "compression.type": "zstd",
})

def on_delivery(err, msg):
    if err:
        print("fail:", err)
    else:
        print(msg.topic(), msg.partition(), msg.offset())

p.produce(
```

★ producer tips that save you at 3am

- acks='all' + idempotence → no duplicates, no lost writes (set enable.idempotence=True).
- Use a meaningful key so related events land in the same partition (preserves order).
- linger_ms + batch.size = throughput. Don't be afraid of 5-50ms linger.
- Compression: 'zstd' or 'lz4' — almost always a win for JSON.
- Always call flush() before exit, or messages sit in memory forever.
- Wrap send() in try/except — broker can be unavailable, payload too large, etc.

■ delivery guarantees, visualized

acks=0

fire & forget

fastest · may lose data

■■■■■ speed

acks=1

leader ack

fast · loses on leader fail

■■■ ■■■ speed

acks=all

in-sync replicas

safest · slightly slower

■■ ■■■ ■■■ speed

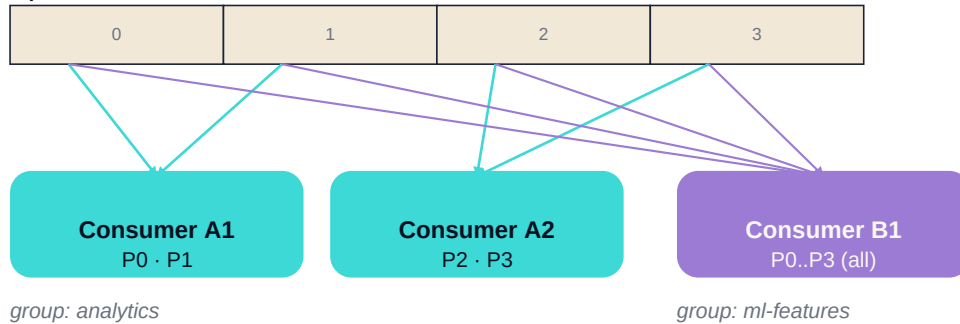


Consumers

read events, manage offsets, scale with consumer groups

consumer groups = parallel reading

topic: clicks



rule of thumb

consumers in a group ≤ # partitions
extra consumers sit idle.

want more parallelism?

→ add more partitions to the topic.

different group = independent replay.

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    "orders",
    bootstrap_servers="localhost:9092",
    group_id="order-pipeline",
    auto_offset_reset="earliest",
    enable_auto_commit=False,
    value_deserializer=lambda b: json.loads(b)
)

for msg in consumer:
    handle(msg.value)
    consumer.commit() # commit AFTER work
```

```
import asyncio, json
from aiokafka import AIOKafkaConsumer

async def main():
    c = AIOKafkaConsumer(
        "orders",
        bootstrap_servers="localhost:9092",
        group_id="order-pipeline",
        value_deserializer=lambda b: json.loads(b)
    )
    await c.start()
    try:
        async for msg in c:
            await handle(msg.value)
            await c.commit()
    finally:
        await c.stop()
```

offsets, committed vs current



■ committed offset (6)
 ■ current position (7)
 ■ already processed
 ■ not yet read

! consumer pitfalls (don't be that person)

- Auto-commit can ack messages you haven't actually processed → set `enable_auto_commit=False`.
- A long handler can trigger session timeout & rebalance → keep work fast or use `max.poll.interval.ms`.
- Always handle deserialization errors. Bad bytes happen. Use a dead-letter topic.



Patterns & Ops

real-world snippets, CLI, debugging, and the survival kit

```
from kafka.admin import KafkaAdminClient, NewTopic

admin = KafkaAdminClient(
    bootstrap_servers="localhost:9092",
)

admin.create_topics([
    NewTopic(
        name="orders",
        num_partitions=6,
```

```
# list topics
kafka-topics --bootstrap-server :9092 --list

# create
kafka-topics ... --create --topic t \
    --partitions 6 --replication-factor 3

# describe (shows partitions/leaders)
kafka-topics ... --describe --topic t
```

```
from confluent_kafka import Producer, Consumer, TopicPartition

p = Producer({
    "bootstrap.servers": "localhost:9092",
    "enable.idempotence": True,
    "transactional.id": "orders-tx-1",
    "acks": "all",
})
p.init_transactions()

c = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "orders-etl",
```

◆ tuning cheatsheet

throughput	linger.ms ↑, batch.size ↑, compression
latency	linger.ms 0, smaller batches
durability	acks=all, idempotence, RF ≥ 3
ordering	use a stable message key
rebalance pain	use cooperative-sticky assignor
memory blow-up	max.poll.records ↓, max.partition.fetch.by

★ when things go wrong

- ConsumerGroup is rebalancing → check session.timeout.ms & long
- 'Leader not available' → broker just started, retry. Check --describe.
- Lag growing forever → consumer too slow, scale group or add partiti
- Out-of-order events → you're using random keys; pick a stable key.
- Duplicates after retry → enable idempotence, or use transactions.
- 'RecordTooLarge' → bump message.max.bytes broker + producer si



you got this.

start small: 1 broker, 1 topic, 1 producer, 1 consumer. then scale up.

→ your kafka journey

